

Активность: карта атакуемой поверхности

1. Четыре понятия, с которыми мы работаем

Атакуемая поверхность — это совокупность всех точек, через которые внешний мир может воздействовать на приложение. Чтобы описать её для одного экрана, достаточно четырёх понятий:

- **Точки входа** — всё, через что данные или команды попадают в систему: поля ввода, кнопки и действия, параметры URL, скрытые поля формы, запросы к API, загрузка файлов, куки-файлы и токены, сторонние виджеты.
- **Активы** — то ценное, что обрабатывается на экране. Отдельно данные (персональные данные, пароли, оценки, платежи, токены) и функции (сменить роль, скачать отчёт, оформить заказ).
- **Границы доверия** — места, где данные переходят между сторонами с разным уровнем доверия: браузер <-> сервер <-> база данных <-> внешние сервисы. Главный вопрос: что здесь приходит от клиента и потому не должно приниматься на веру?
- **Злоупотребления** — сценарии использования не по назначению. Формулируются как «действующее лицо → действие → нежелательный эффект».

2. Процесс работы (15 минут в группе)

Шаг	Время	Что делаете
Роли	1 мин	Договоритесь: кто держит время, кто фиксирует результаты, кто представит результат (90 секунд).
Разведка	3 мин	Мысленно пройдите по полученной странице. Перечислите все точки входа — не пропускайте невидимые (API, URL, токены, загрузка файлов).
Активы и доверие	3 мин	Что ценного за этим стоит? Где граница браузер <-> сервер? Что приходит от клиента?
Злоупотребления	5 мин	Пройдите 5 линз (раздел 4). По каждой значимой точке входа — один сценарий «что, если...?».
Топ-риск	2 мин	Выберите самый опасный сценарий и одну серверную меру, которая его закрывает.
Проверка	1 мин	Пробегитесь по чек-листу самопроверки (раздел 6).

3. Канва: «Карта атакуемой поверхности»

Заполните для той страницы, которую получила ваша группа. **Одна канва = один экран.**

Страница / экран:	Juice Shop — Customer Complaint / Обратная связь (/#/complain)
Группа / приложение:	OWASP Juice Shop

Блок канвы	Ваши ответы
1. Точки входа <i>Поля, кнопки/действия, параметры URL, скрытые поля, запросы к API, загрузка файлов, куки-файлы/токены, сторонние виджеты.</i>	• Текстовое поле — сообщение (comment). HTML-теги рендерятся → XSS. • Загрузка файла — поле вложения (invoice). Принимает .pdf, .zip. • API: POST /api/Complaints — требует JWT, принимает JSON. • API: POST /api/Feedbacks — без авторизации (UserId=null), требует captchaId. • API: POST /file-upload — multipart/form-data, возвращает 204. • Скрытый параметр: captchaId — обязателен для /api/Feedbacks. • Параметр: UserId — можно подставить чужой → forged feedback. • Параметр: rating — клиент не даёт 0, сервер принимает 0. • Заголовок: Authorization (JWT) — для /api/Complaints. • Структурированные файлы: XML → XXE (чтение /etc/passwd, DoS). • Структурированные файлы: YAML → Billion Laughs (memory exhaustion). • Устаревший, но доступный путь загрузки (возможно, /ftp/).
2. Активы: данные и функции <i>Отдельно ценные данные и отдельно ценные функции этого экрана.</i>	Данные: • Обращения пользователей (комментарии, рейтинги) — доступны всем через GET /api/Feedbacks. • Привязка обращения к UserId — персональные данные. • Загруженные файлы (счета, документы) — хранятся на сервере. • Системные файлы сервера (/etc/passwd) — достижимы через XXE. Функции: • Отправка жалобы / обратной связи. • Загрузка и хранение вложений.

	- Парсинг структурированных файлов (XML, YAML). - Просмотр и удаление отзывов (feedbackChallenge).
3. Роли и границы доверия <i>Кто легитимно пользуется экраном; где граница браузер <-> сервер <-> БД; что приходит от клиента.</i>	Роли: - Аноним (без авторизации) — отправляет feedback, просматривает отзывы. - Авторизованный пользователь — отправляет complaints с вложениями. - Администратор — просматривает все обращения (жертва XSS). Границы доверия: - Браузер → Сервер: comment, rating, файл. Всему нельзя верить — comment даёт XSS, rating не валидирован, файл может быть любым. - Браузер → Сервер: UserId в теле запроса — клиент подставляет любой. - Браузер → Сервер: captchaId — автоматизируется (10 отзывов за 20 сек). - Сервер → XML-парсер: внешние сущности (XXE). - Сервер → YAML-парсер: якоря-бомбы (Billion Laughs). - Сервер → Файловая система: имя файла — null-байт, двойное расширение.
4. Возможные злоупотребления <i>По каждой значимой точке входа: «действующее лицо → действие → нежелательный эффект».</i>	- Чужой объект: Пользователь → подставляет чужой UserId в POST /api/Feedbacks → пишет отзыв от имени другого. - Чужой объект: Пользователь → подставляет чужой id в GET /api/Feedbacks/N → читает чужие обращения. - Не тот, за кого выдаёт: Аноним → вызывает POST /api/Complaints без токена → доступ к защищённому функционалу. - Ввод → код: Злоумышленник → <script> в comment → XSS у админа. - Ввод → код: Злоумышленник → XML с <!ENTITY ххе SYSTEM "file:///etc/passwd"> → читает системный файл. - Ввод → код: Злоумышленник → XML с /dev/urandom → сервер зависает (XXE DoS). - Ввод → код: Злоумышленник → YAML-бомба → парсер падает по памяти. - Клиенту верить нельзя: Злоумышленник → rating:0 в обход интерфейса → нулевой рейтинг. - Клиенту верить нельзя: Злоумышленник → .exe через перехват → сервер принимает не-PDF/ZIP. - Клиенту верить нельзя: Злоумышленник → файл >100 kB → обход ограничения размера. - Забытое и лишнее: Злоумышленник → старый endpoint загрузки → обход новых проверок. - Забытое и лишнее: Злоумышленник → file.pdf%00.exe → обход проверки расширения (null-байт).
5. Топ-1 риск и первая защита <i>Самый опасный сценарий + одна серверная мера, которая его</i>	Риск: XXE-инъекция через загруженный XML-файл — злоумышленник читает /etc/passwd и другие системные файлы, а также может вызвать отказ в обслуживании (XXE DoS).

закрывает.

Первая защита: Отключить обработку внешних сущностей (DTD) в XML-парсере на сервере:

```
parser.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);  
parser.setFeature("http://xml.org/sax/features/external-general-entities", false);  
parser.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
```

Также отказаться от парсинга структурированных форматов (XML, YAML) из непроверенных источников, либо использовать безопасные парсеры (defusedxml для Python, DefusedXML для Node.js).

4. Пять линз для поиска злоупотреблений

Если группа застряла — прогоните каждую точку входа через эти вопросы. Знать названия уязвимостей не нужно: достаточно здравого смысла.

- **Линза «Чужой объект»**
Если в запросе или в URL есть чей-то идентификатор (id заказа, корзины, пользователя, работы) — что будет, если подставить чужой?
- **Линза «Не тот, за кого себя выдаёт»**
Можно ли попасть в функцию без нужной роли? Доверяем ли роли или признаку, который пришёл от клиента (куки-файлы, токен, скрытое поле)?
- **Линза «Ввод, который станет кодом»**
Куда попадает моё поле дальше — в запрос к базе, в HTML-страницу, в имя файла, в системную команду? Что если ввести туда не текст, а конструкцию?
- **Линза «Клиенту верить нельзя»**
Какие проверки существуют только в браузере (цена, количество, тип файла, обязательность поля)? Что будет, если обратиться к API напрямую, мимо интерфейса?
- **Линза «Забытое и лишнее»**
Есть ли старые или отладочные адреса, служебные файлы, подробные сообщения об ошибках, включённые по умолчанию функции?

Подсказка

Хорошее злоупотребление всегда содержит трёх участников фразы: кто действует, что именно делает и какой вред получается. «Студент подставляет чужой id в адрес и видит чужие оценки» — годится. «Тут возможен взлом» — нет.

5. Заполненный пример

Страница: LMS — «Мои оценки / ведомость». Это другой экран, такого нет у вас в карточках, — он показывает, как примерно должна выглядеть готовая канва.

Блок канвы	Пример заполнения
1. Точки входа	• Параметр в адресе с идентификатором студента и курса (например, /grades?studentId=1042). • Фильтры по семестру и курсу (параметры запроса). • Кнопка «Экспорт в PDF / CSV». • Фоновый запрос к API, который тянет оценки (виден в DevTools → Network). • Токен сессии в заголовке запроса. • Для преподавателя — форма выставления и редактирования оценки.
2. Активы: данные и функции	• Данные: оценки, ФИО, номер зачётной книжки, связь «студент-курс» — персональные и учебные данные. • Функции: просмотр ведомости, экспорт, у преподавателя — изменение оценки.
3. Роли и границы доверия	• Роли: студент (только свои оценки), преподаватель (свои курсы), деканат/админ (все). • Граница браузер <-> сервер: идентификатор студента приходит от клиента — доверять нельзя. • От клиента также приходят courseId и роль внутри токена — их тоже нельзя принимать на веру.
4. Возможные злоупотребления	• Студент меняет studentId=1042 на 1043 и видит чужие оценки (доступ к чужому объекту). • Пользователь без роли «преподаватель» напрямую вызывает API редактирования оценки. • Экспорт возвращает больше строк, чем показано в интерфейсе (утечка через отчёт). • Подмена роли в токене или скрытом поле открывает ведомость всего курса.
5. Топ-1 риск и первая защита	• Риск: просмотр и изменение чужих оценок через подстановку идентификатора при отсутствии серверной проверки прав. • Первая защита: авторизация на сервере по каждому объекту — сверять запрашиваемые studentId и courseId с ролью и правами текущего пользователя, а не полагаться на интерфейс и клиентские параметры.

6. Чек-лист самопроверки (перед выступлением)

- Мы перечислили не только видимые поля, но и API, параметры URL, загрузку файлов и токены.
- Мы отделили данные от функций и назвали действительно ценное.
- Мы явно указали, что приходит от клиента и чему нельзя верить.
- Каждое злоупотребление сформулировано как «действующее лицо → действие → эффект».
- Наша защита — серверная и конкретная (не «поставить firewall» и не «обучить пользователей»).
- Мы уложимся в 90 секунд и знаем, кто говорит.